

INTRODUCTION TO PROGRAMMING (CSE1001)
• MODULE 2

Arrays and Functions

Arrays: declaration, initialization, accessing elements- Array manipulation. Functions: The prototype declaration, Function definition, Function call: Passing arguments to a function, by value, by reference. Scope of variable names. Recursive function calls.

By Ms Savathallapalli Jasmine

Declaring an array

An array is a collection of elements of the **same data type**, stored in **contiguous memory** and accessed using an index starting at 0.

Syntax: `datatype arrayName [ size ] ;`

- size must be a positive integer constant (or a constant expression)
- elements are indexed from 0 to size-1
- all elements occupy one contiguous block of memory
- an uninitialized local array holds garbage values

```
// Declaring arrays
int marks[5];           // 5 integers
float price[10];        // 10 floats
char name[20];          // 20 characters

// Memory layout of marks[5]
// index:  0      1      2      3      4
// value: [--] [--] [--] [--] [--]

// 2-D array declaration
int matrix[3][3]; // 3 rows, 3 columns
```

C

Ways to initialize an array

c

```
// 1. Full initialization
int arr[5] = {10, 20, 30, 40, 50};

// 2. Size inferred from values -> size 3
int arr2[] = {1, 2, 3};

// 3. Partial init -> remaining elements = 0
int arr3[5] = {1, 2};           // {1,2,0,0,0}

// 4. All elements set to 0
int arr4[5] = {0};

// 5. 2-D array initialization
int matrix[2][3] = {{1,2,3}, {4,5,6}};
```

Rules

- You cannot exceed the declared size with more initializers — that's a compile error.
- If the size is omitted, the compiler counts the initializers.
- Leftover elements in a partial initialization are auto-set to 0.
- A global/static array with no initializer is zero-filled automatically; a local one is not.

Accessing array elements

Each element is reached with **arrayName[index]**. Valid indices run from 0 to n-1; going outside that range is **undefined behaviour** in C — the compiler will not stop you.

arr[2] = 30

index	0	1	2	3	4
value	10	20	30	40	50

^
arr[2] = 30

Common patterns

Reading: `int x = arr[i];`

Writing: `arr[i] = 99;`

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i < 5; i++) {
        printf("arr[%d] = %d\n", i,
arr[i]);
    }
    return 0;
}

/* Output:
arr[0] = 10
arr[1] = 20 ... arr[4] = 50 */
```

Finding the Address of Array Elements

- **Formula:**

- **Address of arr[i] = Base Address + (i × Size of one element)**

- **Example:**

- `int arr[5] = {10, 20, 30, 40, 50};`
 - Base Address = 1000
 - `sizeof(int)` = 4 bytes
 - Address calculation for arr[3]
 - Address of arr[3] = $1000 + (3 * 4) = 1012$

Memory Representation

- Visual representation of contiguous memory locations
- Addresses:
 - arr[0] = 1000
 - arr[1] = 1004
 - arr[2] = 1008
 - arr[3] = 1012
 - arr[4] = 1016

Types of arrays: 1-D, 2-D & 3-D

Arrays can have more than one "dimension" — think of it as a list, a grid, or a stack of grids.

1-D Array

A single row of values — like one shelf of boxes.

```
int marks[5];

int marks[5] =
    {90, 85, 70, 60, 95};
// access:
marks[2]    // -> 70
```

1D

index:	0	1	2	3	4
value:	90	85	70	60	95

Used for: a list of marks, roll numbers, temperatures, etc.

2-D Array

Rows and columns — like a table or a chess board.

```
int a[2][3];
// 2 rows, 3 columns

int a[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
// access:
a[1][2]    // -> 6
```

2D

[row][col]	row 0:	1	2	3
	row 1:	4	5	6

Used for: matrices, tables, seating charts, tic-tac-toe boards.

Multi Dimensional Array

A stack of 2-D grids — like several tables piled up.

```
int b[2][2][2];
// 2 blocks of 2x2

int b[2][2][2] = {
    {{1,2},{3,4}},
    {{5,6},{7,8}}
};
// access:
b[1][0][1] // -> 6
```

3D

Think: [block][row][col]

Used for: a small cube of data, e.g. RGB image blocks or 3-D grids.

1. Sum of All Elements in a 1D Array

Add up every element of the array using a loop.



```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int sum = 0;

    for (int i = 0; i < 5; i++) {
        sum = sum + arr[i];
    }
    printf("Sum = %d", sum);
    return 0;
}
```

OUTPUT Sum = 150

2. Largest Element in a 1D Array

Assume the first element is the largest, then compare it with the rest.

```

#include <stdio.h>

int main() {
    int arr[5] = {12, 45, 3, 67, 21};
    int large = arr[0];

    for (int i = 1; i < 5; i++) {
        if (arr[i] > large)
            large = arr[i];
    }
    printf("Largest = %d", large);
    return 0;
}
```

OUTPUT Largest = 67

3. Reverse a 1D Array

Swap elements from the two ends of the array, moving toward the centre.



```
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int temp;

    for (int i = 0; i < 5 / 2; i++) {
        temp = arr[i];
        arr[i] = arr[4 - i];
        arr[4 - i] = temp;
    }
    for (int i = 0; i < 5; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

OUTPUT 5 4 3 2 1

4. Sum of All Elements in a 2D Array

Use two nested loops — one for rows, one for columns.

```

#include <stdio.h>

int main() {
    int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int sum = 0;

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            sum += arr[i][j];

    printf("Sum = %d", sum);
    return 0;
}
```

OUTPUT Sum = 21

5. Transpose of a 2D Array

The transpose turns rows into columns: element $[i][j]$ becomes $[j][i]$.

```
#include <stdio.h>

int main() {
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int t[3][2];
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            t[j][i] = a[i][j];

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++)
            printf("%d ", t[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT 1 4 2 5 3 6 (printed on 3 lines)

6. Multiply Two 2D Matrices

For each cell $[i][j]$ in the result, multiply a row of A by a column of B and add the products.

```
#include <stdio.h>

int main() {
    int a[2][2] = {{1, 2}, {3, 4}};
    int b[2][2] = {{5, 6}, {7, 8}};
    int c[2][2];

    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++) {
            c[i][j] = 0;
            for (int k = 0; k < 2; k++)
                c[i][j] += a[i][k] * b[k][j];
        }

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++)
            printf("%d ", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

OUTPUT 19 22 43 50 (printed on 2 lines)

Array Manipulation

Common operations: traversal, insertion, deletion, searching, sorting

- Traversal — visiting each element (sum, average, max/min)
- Insertion — shifting elements to make room for a new value
- Deletion — shifting elements left to close a gap
- Searching — linear or binary search for a target value
- Sorting — arranging elements (e.g., bubble sort) in order

```
// Sum of all elements
int sum = 0;
for (int i = 0; i < 5; i++)
    sum += marks[i];

// Simple linear search for value 92
for (int i = 0; i < 5; i++) {
    if (marks[i] == 92) {
        printf("Found at %d", i);
    }
}
```

Example: find the max and min

Problem: Given an array of 6 integers, print the largest and smallest values using a single pass.

```
#include <stdio.h>
int main() {
    int arr[6] = {12, 45, 3, 67, 22, 9};
    int max = arr[0], min = arr[0];

    for (int i = 1; i < 6; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    printf("Max = %d, Min = %d\n", max, min);
    return 0;
}
```

c

Walkthrough

- 1 Start by assuming the first element is both the max and the min.
- 2 Scan the remaining elements once (i = 1 to 5).
- 3 Update max whenever a bigger value appears; update min whenever a smaller value appears.
- 4 After the loop, max and min hold the correct answers.

Output: Max = 67, Min = 3

Inserting & deleting elements

Arrays have a fixed size, so "inserting" means shifting elements right to make room, and "deleting" means shifting elements left to close the gap.

Insert 25 at position 2 (a simple, complete program)

```
#include <stdio.h>
int main() {
    int arr[6] = {10, 20, 30, 40, 50};
    int n = 5, pos = 2, val = 25;

    for (int i = n; i > pos; i--)
        arr[i] = arr[i - 1]; // shift right
    arr[pos] = val;
    n++;

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
// Output: 10 20 25 30 40 50
```

C

Delete the element at position 2

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int n = 5, pos = 2;

    for (int i = pos; i < n - 1; i++)
        arr[i] = arr[i + 1]; // shift left
    n--;

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
// Output: 10 20 40 50
```

C

Searching & sorting an array

Linear search — $O(n)$

```
int search(int arr[], int n, int key) {  
    for (int i = 0; i < n; i++)  
        if (arr[i] == key)  
            return i;    // found: return index  
    return -1;           // not found  
}
```

C

Each pass of bubble sort "bubbles" the largest remaining value to the end — after $n-1$ passes the whole array is sorted in ascending order.

Bubble sort — $O(n^2)$

```
void bubbleSort(int arr[], int n) {  
    for (int i = 0; i < n - 1; i++)  
        for (int j = 0; j < n-i-1; j++)  
            if (arr[j] > arr[j + 1]) {  
                int t = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = t;  
            }  
}
```

C

What is a String?

- ◆ A string is simply a group (array) of characters.
- ◆ In C, a string is stored as an array of type char.
- ◆ Every string ends with a special hidden character called the null character '\0'.
- ◆ The '\0' tells the compiler "the string ends here".
- ◆ Example: the word "Hello" needs 6 boxes in memory, not 5 — the last one is '\0'.



Memory layout of char name[6] = "Hello";

```
#include <stdio.h>
int main() {
    char name[6] = "Hello";
    printf("%s", name);
    return 0;
}
```

Declaring and Initializing Strings

There are three common ways to create a string in C.

- ◆ Method 1 — Give a size and a value:
◆ `char str[20] = "Hello";`
- ◆ Method 2 — Let the compiler count the size:
◆ `char str[] = "Hello";`
- ◆ Method 3 — Character by character:
◆ `char str[6] = {'H','e','l','l','o','\0'};`
- ◆ Tip: Always leave room for one extra box for `'\0'`.

Example Program

```
#include <stdio.h>
int main() {
    char str1[20] = "Hello";
    char str2[] = "World";
    char str3[6] =
        {'H','e','l','l','o','\0'};

    printf("%s %s\n", str1, str2);
    printf("%s", str3);
    return 0;
}
```

Reading a String (Input)

Three common ways to take string input from the user.

- ◆ `scanf("%s", str);`
- ◆ Easy to use, but stops reading at the first space.
- ◆ `gets(str);`
- ◆ Reads a full line including spaces, but it is unsafe and outdated — avoid using it.
- ◆ `fgets(str, size, stdin);`
- ◆ Safe and recommended — reads a full line including spaces.

Example Program

```
#include <stdio.h>
int main() {
    char name[30];
    printf("Enter your name: ");
    fgets(name, 30, stdin);
    printf("Hello, %s", name);
    return 0;
}
```

Printing a String (Output)

Two simple ways to display a string on the screen.

- ◆ `printf("%s", str);`
- ◆ Most common way. The `%s` format specifier is used for strings.
- ◆ `puts(str);`
- ◆ Simpler function made only for printing strings. It also adds a new line automatically.

Example Program

```
#include <stdio.h>
int main() {
    char str[] = "Welcome to C!";
    printf("%s\n", str);
    puts(str);
    return 0;
}
```

The <string.h> Library — All Common Operations

C provides many ready-made string functions — just include <string.h>.

strlen(str)

Finds the length of a string

strcmp(str1, str2)

Compares two strings

strcpy(dest, src)

Copies one string into another

strncmp(str1, str2, n)

Compares only the first n characters

strncpy(dest, src, n)

Copies only the first n characters

strchr(str, ch)

Finds the first occurrence of a character

strcat(str1, str2)

Joins (concatenates) two strings

strstr(str1, str2)

Finds a substring inside a string

strncat(str1, str2, n)

Joins only the first n characters

strtok(str, delim)

Splits a string into tokens (pieces)

Note: tolower() and toupper() change character case but come from <ctype.h>, not <string.h>.

strlen() — Find the Length

Counts the number of characters in a string, not including the '\0'.



```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello";
    int len = strlen(str);
    printf("Length = %d", len);
    return 0;
}
```

OUTPUT Length = 5

strcpy() — Copy a String

Copies the contents of one string into another string.



```
#include <stdio.h>
#include <string.h>

int main() {
    char src[] = "Hello";
    char dest[20];

    strcpy(dest, src);
    printf("Copied string = %s", dest);
    return 0;
}
```

OUTPUT Copied string = Hello

strcat() — Join Two Strings

Adds (concatenates) the second string to the end of the first string.

```

● ● ●
#include <stdio.h>
#include <string.h>

int main() {
    char str1[20] = "Hello ";
    char str2[] = "World";

    strcat(str1, str2);
    printf("Result = %s", str1);
    return 0;
}
```

OUTPUT Result = Hello World

strcmp() — Compare Two Strings

Compares two strings and returns 0 if equal, negative if $\text{str1} < \text{str2}$, positive if $\text{str1} > \text{str2}$.



```
#include <stdio.h>
#include <string.h>

int main() {
    char a[] = "cat";
    char b[] = "dog";

    int result = strcmp(a, b);
    printf("Result = %d", result);
    return 0;
}
```

OUTPUT Result = -1 (since 'c' comes before 'd')

strncpy(), strncat(), strncmp() — Safer 'n' Versions

These work like strcpy/strcat/strcmp but only touch the first n characters — safer against overflow.



```
#include <stdio.h>
#include <string.h>

int main() {
    char dest[20] = "";
    strncpy(dest, "Hello World", 5);
    dest[5] = '\0';
    printf("strncpy  = %s\n", dest);

    strncat(dest, "!!!", 2);
    printf("strncat  = %s\n", dest);

    int r = strncmp("Hello", "Help", 3);
    printf("strncmp  = %d\n", r);
    return 0;
}
```

OUTPUT strncpy = Hello strncat = Hello!! strncmp = 0

strchr() — Find a Character in a String

Searches for the first occurrence of a character and returns a pointer to it.

```

• • •
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello World";
    char *pos = strchr(str, 'W');

    if (pos != NULL)
        printf("Found at position %d", (int)(pos - str));
    else
        printf("Not found");
    return 0;
}
```

OUTPUT Found at position 6

strstr() — Find a Substring

Searches for a whole substring inside a string and returns a pointer to where it starts.

```
• • •  
  
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char str[] = "Learning C Programming";  
    char *pos = strstr(str, "C Prog");  
  
    if (pos != NULL)  
        printf("Substring found: %s", pos);  
    else  
        printf("Not found");  
    return 0;  
}
```

OUTPUT Substring found: C Programming

strtok() — Split a String into Tokens

Breaks a string into smaller pieces (tokens) wherever a delimiter character appears.



```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Name, Age, City";
    char *token = strtok(str, ",");

    while (token != NULL) {
        printf("%s\n", token);
        token = strtok(NULL, ",");
    }
    return 0;
}
```

OUTPUT Name Age City (printed on 3 lines)

Convert a String to Lowercase

We use `tolower()` from `<ctype.h>` on every character, since `strlwr()` is not a standard function.



```
#include <stdio.h>
#include <ctype.h>

int main() {
    char str[] = "HELLO WORLD";
    for (int i = 0; str[i] != '\0'; i++) {
        str[i] = tolower(str[i]);
    }
    printf("Lowercase = %s", str);
    return 0;
}
```

OUTPUT Lowercase = hello world

Convert a String to Uppercase

Same idea as before, but using `toupper()` from `<ctype.h>` instead.



```
#include <stdio.h>
#include <ctype.h>

int main() {
    char str[] = "hello world";
    for (int i = 0; str[i] != '\0'; i++) {
        str[i] = toupper(str[i]);
    }
    printf("Uppercase = %s", str);
    return 0;
}
```

OUTPUT Uppercase = HELLO WORLD

Reverse a String

Swap characters from both ends of the string, moving toward the middle.



```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello";
    int len = strlen(str);
    char temp;

    for (int i = 0; i < len / 2; i++) {
        temp = str[i];
        str[i] = str[len - 1 - i];
        str[len - 1 - i] = temp;
    }
    printf("Reversed = %s", str);
    return 0;
}
```

OUTPUT Reversed = olleH

count vowels

Counting vowels

```
int countVowels(char str[]) {  
    int count = 0;  
    for (int i = 0; str[i] != '\0'; i++) {  
        char c = str[i];  
        if (c=='a' || c=='e' || c=='i' ||  
            c=='o' || c=='u')  
            count++;  
    }  
    return count;  
}  
// "hello" -> 2
```

c

Tip: both reversing a string and counting vowels patterns walk the string with an index until they hit '\0' (or two indices closing in from each end). Almost every string problem — case conversion, removing spaces, finding duplicates — reuses this same scan pattern.

Bonus: Check if a String is a Palindrome

A palindrome reads the same forwards and backwards, e.g. "madam". Compare characters from both ends.



```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "madam";
    int len = strlen(str);
    int isPalin = 1;

    for (int i = 0; i < len / 2; i++) {
        if (str[i] != str[len - 1 - i]) {
            isPalin = 0;
            break;
        }
    }
    if (isPalin)
        printf("%s is a Palindrome", str);
    else
        printf("%s is NOT a Palindrome", str);
    return 0;
}
```

OUTPUT madam is a Palindrome

Why use functions?

1

Reusability

Write the logic once and call it from anywhere in the program.

2

Modularity

Break a large problem into small, manageable, testable pieces.

3

Readability

Well-named functions make the program's intent obvious.

4

Maintainability

Fix or improve logic in one place instead of many.

Anatomy of a function

```
returnType functionName(parameterList) {  
    // function body  
    return value;    // optional, matches returnType  
}
```

```
int  add    (int a, int b) { return a + b; }  
^ return   ^name ^parameters   ^return  
type
```

C

Library Functions vs User-Defined Functions

Every function in C belongs to one of these two kinds.

Library Functions

`printf()`, `scanf()`, `strlen()`, `sqrt()`

- ◆ Already written and tested — part of C's standard library.
- ◆ We simply call them by name to use them.
- ◆ We must include the correct header file, e.g. `<stdio.h>`, `<string.h>`, `<math.h>`.
- ◆ We cannot see or edit their internal code.

User-Defined Functions

`sum()`, `factorial()`, `square()`, `swap()`

- ◆ Written by the programmer to do a specific task.
- ◆ Must be declared and defined before they are used.
- ◆ We can read, change, and reuse this code anywhere.
- ◆ Help break a big problem into smaller, manageable parts.

Using functions avoids repeating code, makes programs easier to read, and makes debugging simpler.

4 Types of User-Defined Functions

Based on whether a function takes arguments and whether it returns a value.

1. No arguments, no return value

```
void greet() {  
    printf("Hello!");  
}
```

2. No arguments, with return value

```
int getNum() {  
    return 5;  
}
```

3. With arguments, no return value

```
void greet(char n[]) {  
    printf("Hi %s", n);  
}
```

4. With arguments, with return value

```
int add(int a, int b) {  
    return a + b;  
}
```

Prototype declaration vs. definition

Prototype (declaration)

Tells the compiler the function's name, return type and parameter types before it is used — so the function body can appear later, even in another file.

```
// Prototype: ends with a semicolon,  
// no body. Usually placed above main()  
// or in a header (.h) file.  
int add(int a, int b);  
float average(float x, float y);  
void printMessage(void);
```

c

Definition

The actual body of the function: the real code that executes when the function is called. Written once, anywhere in the file.

```
// Definition: has a body in { }  
int add(int a, int b) {  
    return a + b;  
}
```

c

```
#include <stdio.h>  
int add(int a, int b);           // 1. prototype  
int main() { printf("%d", add(5, 3)); return 0; } // 2. call  
int add(int a, int b) { return a + b; }          // 3. definition
```

c

Calling a function

A function call transfers control to the function's code, executes its body, and (if it returns a value) sends a result back to the point of the call.



Arguments vs. parameters

Arguments — the actual values sent in the call: `add(5, 3)`

Parameters — the placeholder names in the definition: `add(int a, int b)`

```
#include <stdio.h>
int add(int a, int b);

int main() {
    int result = add(5, 3); // function call
    printf("Sum = %d", result);
    return 0;
}

int add(int a, int b) { return a + b; }
```

Passing arguments: by value

In call-by-value, the function receives a copy of the argument. Changes made inside the function affect only that copy — the caller's original variable is untouched.

- The default way arguments are passed in C.
- Safe: the function can never accidentally modify the caller's data.
- Slightly costlier for large data (a copy is made) — fine for simple types

```
main's a = 5
      |   copies value
      v
modify's x = 5 -> x = 15 (local only)

back in main(): a is STILL 5
```

```
#include <stdio.h>
void modify(int x) {
    x = x + 10;      // only the local
copy
}                  // changes

int main() {
    int a = 5;
    modify(a);
    printf("%d", a); // prints 5
(unchanged)
    return 0;
}
```

Passing arguments: by reference

C has no native reference type, so "by reference" is simulated with pointers: the function receives the address of the variable and can modify the original directly.

- Pass the address with & ; receive it into a pointer parameter.
- Dereference with * inside the function to read/write the original value.
- Used whenever a function must return more than one value, or must modify the caller's data (e.g. swap, array-size updates).

Classic use case: swap()

```
void swap(int *a, int *b) {  
    int temp = *a;  
    *a = *b;  
    *b = temp;  
}  
// call as: swap(&x, &y);
```

C

```
#include <stdio.h>  
  
void modify(int *x) {  
    *x = *x + 10;    // changes the value  
}                  // AT the original  
address  
  
int main() {  
    int a = 5;  
    modify(&a);      // pass address of a  
    printf("%d", a); // prints 15 (changed)  
    return 0;  
}
```

C

Call by value vs. call by reference

Aspect	Call by Value	Call by Reference
What is passed	A copy of the argument's value	The address (pointer) of the argument
Can modify caller's data?	No — original stays unchanged	Yes — original can be updated
Parameter syntax	<code>void f(int x)</code>	<code>void f(int *x)</code>
Call syntax	<code>f(a);</code>	<code>f(&a);</code>
Access inside function	<code>x</code>	<code>*x</code>
Typical use	Simple read-only inputs	<code>swap()</code> , updating multiple results, large data

Scope of variable names

Local scope

Declared inside a function/block. Exists only while that function runs; invisible outside it.

```
int localVar = 5; // inside a
function
```

Global scope

Declared outside all functions. Visible to every function in the file, from declaration to end of file.

```
int globalVar = 100; // outside
main()
```

Static (local)

A local variable declared static retains its value between successive calls instead of resetting.

```
static int count = 0;
```

```
#include <stdio.h>
int globalVar = 100;           // global scope: visible everywhere below
void display() {
    static int count = 0;      // static: keeps its value across calls
    int localVar = 5;         // local scope: fresh every call, then gone
    count++;
    printf("call #%d, global=%d\n", count, globalVar);
}
int main() {
    display(); // call #1, global=100
    display(); // call #2, global=100 <-- count kept its value
    display(); // call #3, global=100
    return 0;
}
```

C

How recursive function calls work

A **recursive function** calls itself to solve a smaller version of the same problem, until it reaches a **base case** that stops the recursion.

- 1 Base case** — the simplest input, answered directly without another call — stops infinite recursion
- 2 Recursive case** — the function calls itself with a smaller / simpler argument, moving toward the base case
- 3 Call stack** — each call waits on the stack until the one below it returns, then the results combine on the way back up

factorial(4) unwinding the call stack

```
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1 * factorial(0)
factorial(0) = 1                <- base case
returns 1 * 1 = 1
returns 2 * 1 = 2
returns 3 * 2 = 6
returns 4 * 6 = 24
```

[trace](#)

1. Sum of Two Numbers Using a Function

A function named `sum()` takes two numbers and returns their total.



```
#include <stdio.h>

int sum(int a, int b) {
    return a + b;
}

int main() {
    int x = 10, y = 20;
    printf("Sum = %d", sum(x, y));
    return 0;
}
```

OUTPUT Sum = 30

2. Factorial Using a Recursive Function

A recursive function calls itself with a smaller value until it reaches the base case ($n == 0$).



```
#include <stdio.h>

int factorial(int n) {
    if (n == 0)
        return 1;
    return n * factorial(n - 1);
}

int main() {
    printf("Factorial = %d", factorial(5));
    return 0;
}
```

OUTPUT Factorial = 120

3. Square of a Number Using a Function

A function named `square()` takes one number and returns its square.



```
#include <stdio.h>

int square(int n) {
    return n * n;
}

int main() {
    int num = 7;
    printf("Square = %d", square(num));
    return 0;
}
```

OUTPUT Square = 49

4. Swap Two Numbers Using a Function

Swapping needs pointers (call by reference), so the changes affect the original variables.

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    swap(&x, &y);
    printf("x = %d, y = %d", x, y);
    return 0;
}
```

OUTPUT x = 10, y = 5

Factorial & Fibonacci

Factorial (n!)

```
int factorial(int n) {  
    if (n == 0)  
        return 1;           // base case  
    return n * factorial(n - 1); // recursive  
case  
}  
  
// factorial(5) = 5*4*3*2*1*1 = 120
```

C

Fibonacci sequence

```
int fibonacci(int n) {  
    if (n == 0) return 0;    // base case 1  
    if (n == 1) return 1;    // base case 2  
    return fibonacci(n - 1)  
        + fibonacci(n - 2); // recursive case  
}  
  
// 0, 1, 1, 2, 3, 5, 8, 13, 21 ...
```

C

Recursion vs. iteration

- Recursion often mirrors the mathematical definition directly (e.g. $n! = n \times (n-1)!$) and can be more readable for tree-like or divide-and-conquer problems.
- Every call uses stack memory, so deep or missing base cases can cause a stack overflow — always guarantee the recursion moves toward the base case.
- An iterative loop typically uses less memory and is often preferred for simple counting problems like factorial.

TEST YOURSELF

Practice Questions

- 1 Write a C program to find the second largest element in an array without fully sorting it.
- 2 Write a recursive function to compute the GCD (Greatest Common Divisor) of two integers.
- 3 Write a function using call by reference that cyclically swaps the values of three variables.
- 4 Write a C program to check whether two given strings are anagrams of each other.
- 5 Write a program with a static variable inside a function that counts how many times the function has been called across the program's run.